INKKEY

Personal Handwriting Keyboard

Product Requirements Document

Android · Offline-first · Zero-budget MVP

Version 1.0 | 22 October 2026

Implementation-ready product, UX, architecture, and acceptance specification

1. Product summary

InkKey is an Android personalization app whose primary feature is a complete system keyboard that lets users type normally while using a personal handwriting typeface generated from guided, individual-glyph capture. The app also includes a font studio and a renderer that exports handwriting-style text as PNG images. It is not merely a text-to-image toy: the keyboard must be a real Android Input Method Editor (IME), with practical editing behavior and a usable everyday layout.

Product promise

“Type at digital speed. Keep the character of your own handwriting.”

Core experience

1. The user opens InkKey and sees a short, welcoming introduction.
2. The user creates a handwriting profile and completes guided capture sheets.
3. InkKey converts captured glyphs into a font file using a validated, local font-generation pipeline.
4. The user reviews a sample paragraph and adjusts supported typography settings.
5. The user enables InkKey from Android keyboard settings and selects it as their active keyboard.
6. The user types Unicode text into ordinary apps. The keyboard sends standard text; it does not force other apps to use the custom font.
7. When visual handwriting is needed, the user opens the Handwriting Studio, renders text with their font, and shares a PNG.

Non-negotiable constraints

- Zero-budget development and operation: no required paid APIs, subscriptions, hosted backend, or mandatory account.
- Offline-first: capture, font generation, keyboard typing, preview, and image export should work without network access.
- Native Android implementation; do not build a web page that only looks like a keyboard.
- No fake buttons, simulated keyboard activation, placeholder font-generation success, or mock persistence in production flows.
- The app must explain the system-font limitation honestly: normal IME text is Unicode, and recipient apps usually choose their own fonts.

2. Goals, non-goals, and success metrics

Goals

- Generate a valid, usable English handwriting TTF from user-captured glyphs.
- Provide a complete everyday QWERTY keyboard implemented as an Android IME.
- Preserve profile, glyph, font, and settings data across process death and device restart.
- Offer an in-app live font preview and shareable PNG output for apps that cannot render custom fonts.
- Keep personal handwriting and typed content on-device by default.
- Deliver a clean, premium, accessible interface with predictable navigation and polished empty, loading, success, and error states.

Non-goals for v1

- Forcing WhatsApp, Messenger, Instagram, SMS, or other recipient apps to display the custom font when sending ordinary Unicode text.
- Cloud sync, accounts, social feeds, subscriptions, AI handwriting imitation, or remote analytics.
- Bangla, Arabic, Indic, CJK, or other complex-script font generation in the initial release.

- Full word prediction, cloud suggestions, grammar correction, voice dictation, clipboard cloud sync, or a third-party emoji search service.
- Perfect cursive joining or professional-grade automatic kerning in the first release.
- Claiming a handwriting font is a legally valid signature or identity verification.

Metric	MVP target / definition
Capture completion	A new user can finish required glyph capture with progress saved after every glyph.
Font validity	Generated TTF opens in a font parser and renders the supported test corpus without crashing.
Persistence	Profile, captured glyphs, settings, and last valid font survive force-close and relaunch.
Keyboard readiness	User can enable the IME through Android settings and select it using the system input-method picker.
Editing correctness	Letters, spaces, punctuation, shift, backspace, enter, and cursor-aware edits work in tested fields.
Export correctness	PNG has the selected text, font, background, and resolution; it can be shared through Android Sharesheet.
Privacy	Core functionality uses no network; typed text is not logged or transmitted.

3. Target users and key scenarios

Primary users

- People who want their digital communication to feel more personal.
- Students and journalers who want notes or messages rendered in their own handwriting.
- Users who enjoy customization and want a distinctive keyboard identity.
- Users with no budget for paid font tools and who prefer offline privacy.

Key user stories

- As a new user, I want a guided setup so I understand what the keyboard can and cannot do.
- As a user, I want to draw each glyph in a clear cell and correct it before saving.
- As a user, I want to pause capture and resume later without losing work.
- As a user, I want to preview a realistic sample before replacing my current font profile.
- As a user, I want a full keyboard with familiar editing keys, not a demo keyboard.
- As a user, I want to share visually handwritten text with friends even if their apps do not support my font.
- As a privacy-conscious user, I want my handwriting and typed content to remain on my device.

4. Information architecture and navigation

Use a simple bottom navigation with three primary destinations: Home, Studio, and Settings. Keyboard setup is a guided task accessible from Home and the completion screen. Avoid a crowded dashboard or excessive cards.

Destination	Purpose	Primary actions
Home	Status and next step	Continue capture, generate/review font, set up keyboard, open export studio
Capture Studio	Guided glyph collection	Choose sheet, draw, undo, redo, clear, save, skip only where allowed
Font Studio	Inspect and tune font	Preview corpus, adjust supported settings, regenerate, validate, restore previous version

Keyboard setup	Enable and test IME	Open Android input-method settings, show selected/active status where reliably detectable, launch keyboard test
Handwriting Studio	Compose visual text	Type/paste text, choose paper/ink settings, preview, export/share PNG
Settings	Preferences and data control	Theme, haptics, keyboard height, privacy, backup/export, delete profile/data, About

Navigation rules

- Every screen has a clear title, back behavior, and visible primary action.
- Back navigation must not silently discard unsaved strokes or text.
- Long capture flows show progress and allow safe exit/resume.
- Do not show keyboard as “enabled” just because the user tapped a button; direct them to Android settings and verify state where platform APIs allow.
- Home status is derived from real persisted state, not hard-coded progress.

5. Splash, onboarding, and first-run experience

Splash screen

- Show on every cold app launch, using a minimal InkKey mark and wordmark.
- Keep it brief and avoid artificial delays; route to onboarding only when onboarding is incomplete, otherwise Home.
- Use a light appearance by default, with the splash background matching the app's theme to prevent flashes.
- Do not show a fake loading percentage. Only display a progress indicator if a real initialization task is running.

Onboarding

8. Welcome: explain personal handwriting keyboard in one sentence with a polished preview.
9. How it works: guided capture → font generation → keyboard setup → optional image sharing.
10. Expectation setting: explain that ordinary keyboard text remains standard Unicode; visual handwriting must be rendered/exported when the target app does not use custom fonts.
11. Privacy: explain local storage, keyboard sensitivity, and that InkKey does not send typed text to a server.
12. Start: create first handwriting profile.

Provide Skip only for explanatory slides, not for platform disclosures that are necessary to avoid misleading the user. Store onboarding completion persistently. Include a “Replay introduction” action in Settings.

6. Detailed Home screen requirements

- Header: app wordmark, restrained greeting or neutral title, and Settings shortcut.
- Primary profile card: profile name, font status (Not started / Capture in progress / Ready / Needs repair), and a real preview snippet when available.
- Progress indicator based on required glyphs successfully saved; never use invented completion values.
- Primary CTA changes by state: Start handwriting, Continue capture, Review font, or Set up keyboard.
- Secondary actions: Try keyboard, Open handwriting studio, Create another profile (later if needed).
- Show concise explanations for errors, such as incomplete required glyphs or font generation failure.
- Home should remain useful when no profile exists; show a single clear start action rather than empty analytics.

7. Guided handwriting capture

Capture scope

Version 1 captures English Latin letters and common punctuation. Use a configurable glyph manifest so future scripts can add their own character sets without rewriting the capture UI.

Capture group	Required glyphs
Lowercase	a–z
Uppercase	A–Z
Digits	0–9
Basic punctuation	period, comma, question mark, exclamation mark, colon, semicolon, apostrophe, quotation mark, hyphen, parentheses
Optional symbols	ampersand, slash, at sign, plus, equals, percent; unsupported symbols use a documented fallback

Capture screen UI

- Display one large drawing cell with a subtle baseline, x-height guide, ascender guide, and descender guide where appropriate.
- Show the target glyph clearly, plus a small example only when helpful; never trace over the user's drawing.
- Actions: Undo stroke, Redo stroke, Clear, Retry, Save & Next. Disable actions when not applicable.
- Provide a small progress label such as “Lowercase · 8 of 26” and a progress bar tied to saved glyphs.
- Support finger and stylus input; do not require pressure sensitivity.
- Prevent accidental navigation from losing unsaved strokes. Prompt to save or discard when leaving a dirty canvas.
- Save each accepted glyph immediately and update the database transactionally.

Glyph validation

- Reject empty glyphs with a helpful message.
- Detect extremely tiny or out-of-cell strokes and offer retry; avoid overly strict auto-rejection.
- Let users accept unusual handwriting; visual oddity is not necessarily a capture error.
- Allow replacing a saved glyph and retain the prior version until the replacement is accepted.
- Show a confirmation if a glyph is missing, but do not silently substitute a blank glyph.

Capture persistence

Persist stroke points and metadata as each glyph is saved. On relaunch, restore the exact sheet, next missing glyph, and progress. Store capture version and coordinate-system version so future updates can migrate data safely.

8. Font generation engine

Pipeline

13. Load the versioned glyph manifest and fetch the latest accepted stroke data for each required code point.
14. Normalize each glyph into a shared coordinate system while preserving relative proportions.
15. Generate vector outlines from strokes using a deterministic, testable algorithm. Provide controls or sane defaults for stroke width, cap style, and join style.
16. Calculate glyph advance widths and baseline metrics. Avoid forcing every glyph into identical width.
17. Map glyphs to Unicode code points and construct required font tables using a free, documented font-building toolchain.
18. Compile a candidate TTF in app-private temporary storage.
19. Validate that the file parses, contains expected glyph mappings, and renders a test corpus.

20. Only after validation, atomically promote the candidate to the current font. Keep the last known-good font if generation fails.

Engineering decision and risk

A font compiler is not a trivial UI feature. The MVP must prove the end-to-end pipeline early. A practical route is to prototype with free Python font tooling such as FontTools and a vector-outline library, then decide whether to bundle a compatible runtime, implement a native generator, or use a small native library. Verify Android packaging, licensing, performance, and offline behavior before choosing. Do not assume a Python desktop script automatically works on Android.

Font metadata and rendering

- Set family name from the profile, with safe character filtering.
- Include units-per-em, ascender, descender, line gap, character mapping, horizontal metrics, glyph outlines, and basic naming tables.
- Provide fallback behavior for unimplemented characters without crashing or silently corrupting text.
- Use a versioned file name and atomic replacement.
- Expose a clear error state with retry and diagnostic logging that excludes personal text and raw glyph content.

Font settings for MVP

- Stroke thickness: a few discrete safe presets or a slider with a preview.
- Letter spacing: a bounded slider.
- Slant: optional, applied consistently to outlines or rendering.
- Preview size and background are presentation settings and should not modify the source glyph data.
- Regeneration must be explicit when a setting changes the actual font asset.

Realism boundary

One glyph per character creates a font, not a complete handwriting simulation. Repeated letters may look identical, and automatic joining between cursive letters is not guaranteed. Later versions can capture alternate glyphs, implement contextual substitutions, and improve spacing. Do not promise handwriting that is indistinguishable from a fresh human-written page.

9. Complete keyboard requirements (Android IME)

Keyboard purpose

The keyboard is a full system input method. It must work in normal editable text fields in common Android apps and must not be a mock keyboard embedded only in InkKey.

Required layouts

Layout	Required keys / behavior
Alphabetic	QWERTY letters, Shift, Backspace, 123/symbol switch, comma, period, space, Enter, keyboard switch/hide where appropriate
Numbers & symbols	Digits, common punctuation, symbol page switch, return to alphabetic layout
Shifted	Uppercase letters with predictable one-shot shift; optional caps lock via double tap
Editing	Use InputConnection for commitText, deleteSurroundingText, setSelection where supported, and editor action handling
System integration	IME declaration, label, icon, subtype/configuration where needed, enable/choose keyboard guidance
Visual settings	Light/dark theme, key height/density presets, optional

Keyboard behavior

- Use Android InputMethodService and the appropriate manifest/service metadata, including the required input-method XML configuration.
- Use InputConnection APIs rather than attempting to control another app's UI.
- Respect EditorInfo input types and IME actions. Show suitable action key labels for Next, Done, Search, Send, or Go when the host field requests them.
- Do not intercept or store typed text for personalization.
- Do not attempt to display the custom font in the target app's text field; the host app controls rendering.
- Avoid network permission for the keyboard-only core.
- Support password fields safely: no suggestions, learning, clipboard history, or content logging in password input types.
- Provide a reliable way to switch back to another installed keyboard using Android's system picker where available.
- Handle rotation, small screens, split-screen where supported, and system insets without clipped keys.
- Use touch targets suitable for everyday typing, clear pressed states, and accessible contrast.

Keyboard setup flow

21. Show a short explanation of why a system keyboard must be enabled in Android settings.
22. Open the system input-method settings using the appropriate Android intent.
23. Explain that enabling and selecting are two separate steps.
24. Offer a test field inside InkKey after the user returns.
25. If the IME is not selected, show instructions rather than claiming success.
26. Provide a safety and privacy page explaining Android keyboard access.

Important limitation

Typing with InkKey inserts normal Unicode text. It cannot make arbitrary apps render the user's custom TTF, and it cannot control how recipients see the text. Any UI that suggests otherwise is misleading. The handwriting preview/export studio is the supported way to send visual handwriting.

10. Handwriting Studio and image export

Composer

- Large multiline text editor with character count and clear/paste actions.
- Load the currently selected valid handwriting font.
- Live preview with a sensible debounce for long text.
- Paper choices: transparent/blank, lined paper, and a simple warm-paper preset.
- Ink choices: black, blue, and a limited high-contrast palette.
- Adjustable font size, line spacing, margins, and output resolution presets.
- Show a clear fallback notice if the font lacks a character.
- Do not send the text to a server; rendering is local.

Export

- Render to a bitmap at an appropriate resolution without clipping glyph ascenders, descenders, punctuation, or final lines.
- Support PNG export and Android Sharesheet integration.
- Use a user-visible save location or scoped-storage-compatible flow; do not request broad storage access unnecessarily.
- Show preview before sharing and provide success/error feedback based on actual operation results.

- For long content, paginate or offer a long-image mode with sensible memory limits.
- Avoid embedding hidden tracking metadata.

11. Profiles, data model, and persistence

Core entities

Entity	Important fields
HandwritingProfile	id, name, createdAt, updatedAt, active flag, captureVersion, profileStatus
GlyphSample	profileId, codePoint, strokeDataPath or encoded data, revision, acceptedAt, validation flags
FontAsset	profileId, version, filePath, generatedAt, generatorVersion, checksum, validationStatus, isCurrent
UserSettings	theme, haptics, keyHeight, keyPreview, letterSpacing, strokePreset, defaultPaper, defaultInk
ExportRecord (optional)	createdAt, destination metadata only if needed; never store exported text by default

Persistence requirements

- Use Room for relational metadata and DataStore for simple preferences.
- Use app-private files for glyph stroke payloads and TTF files.
- Never store all glyphs as huge base64 strings in a single preferences value.
- Use migrations when schemas change; never reset the database to fix an upgrade.
- Keep a last-known-good font and recover from incomplete generation after process death.
- Handle low storage with a clear error and do not erase existing profiles.
- Offer profile deletion with a confirmation that identifies what will be removed.
- Provide a user-initiated local export/backup option in a later release; backup should be documented and protected.

12. Visual design system and interaction quality

Visual direction

Premium, quiet, editorial, and tactile. The app should feel like a thoughtfully designed writing instrument: warm paper surfaces, ink-like text, subtle separators, restrained accent color, generous whitespace, and crisp typography. The UI should not look like a generic admin dashboard.

- Default theme: light. Support dark theme as an explicit setting and optionally follow system.
- Use a consistent spacing scale (4/8/12/16/20/24/32 dp) and shared component tokens.
- Respect Android system insets and navigation bars; no buttons or cards may extend beyond safe bounds.
- Use consistent corner radii, border weights, icon sizes, and button heights.
- Use motion sparingly: subtle transitions, tactile stroke feedback, and clear state changes.
- Every screen needs loading, empty, error, disabled, and success states as applicable.
- Avoid excessive gradients, oversized decorative illustrations, and redundant cards.
- Use accessible text sizes, semantic labels, contrast, and scalable layouts.
- Test on narrow phones and large font/display settings; avoid clipped content.

Reusable components

- Primary and secondary buttons
- Profile status card
- Progress indicator

- Glyph drawing cell and guide lines
- Font preview specimen
- Keyboard key and keyboard rows
- Inline error / success banner
- Confirmation dialog
- Settings row and segmented controls
- Empty state and retry state

13. Settings, help, and legal/privacy content

- Appearance: Light / Dark / System (default Light).
- Keyboard: haptics, key preview, height preset, theme, setup guide.
- Font: active profile, preview settings, regenerate font, restore last valid version.
- Data: export profile package (if implemented), delete a profile, delete all local data.
- Privacy: describe what the IME can access, local processing, and no typed-text collection/transmission.
- Help: troubleshooting for keyboard not appearing, incomplete glyphs, font generation failures, and missing glyphs.
- About: app version, open-source notices, and licenses for bundled dependencies.

Do not invent a website, support email, company address, or privacy certification. If the app is distributed publicly, provide an accurate privacy policy and Play Store Data safety disclosures based on the actual build and dependencies.

14. Privacy and security

- No account and no network access required for the MVP.
- Do not log typed text, password input, clipboard content, or raw handwriting by default.
- Avoid analytics SDKs and ad SDKs in the zero-budget MVP.
- Keep profile files inside app-private storage by default.
- Use explicit user actions for export/share and explain that exported images may be stored or transmitted by the destination app.
- Use safe file names and validate imported font/profile packages if import is later added.
- Do not request contacts, microphone, location, or broad file access without a real feature need.
- Document the keyboard's capabilities and limitations clearly in onboarding and Settings.

15. Accessibility and localization

- Use content descriptions for icon-only controls and semantic labels for custom drawing controls.
- Support Android font scaling and sufficient contrast.
- Do not rely only on color to show capture status or validation errors.
- Keep touch targets comfortably large and allow keyboard height adjustments.
- Version 1 UI may ship in English, but text resources should be externalized from the start.
- Bangla UI localization can be added independently from Bangla font generation; these are separate projects.
- The font's initial supported script is English Latin; show clear fallback behavior for unsupported scripts.

16. Error states and recovery

Situation	Expected response
Empty glyph	Keep user on the cell; explain that a stroke is required.
Font generation fails	Show a concise explanation, retry option, and preserve

	previous valid font.
Unsupported character	Use a documented fallback in preview/export and indicate the missing glyph when useful.
Storage full	Stop safely, preserve existing files, and explain how to free space.
App killed during generation	On relaunch, discard incomplete temporary output and retain last valid font.
Keyboard not enabled	Show system settings instructions; do not simulate enabled status.
Keyboard loses focus	Recover through normal IME lifecycle without crashing or retaining text.
PNG export fails	Keep the preview and text in memory; explain the error and allow retry.
Profile deleted	Remove its glyph files and fonts safely; do not affect other profiles.

17. Technical architecture and implementation notes

Suggested Android structure

Use Kotlin, Jetpack Compose for app screens, Room, DataStore, Android Canvas/Path for drawing, InputMethodService for the keyboard, and Android graphics/text APIs for preview and PNG rendering. Keep modules separated so font generation can be replaced without rewriting the UI.

- app: application entry point, navigation, theme, onboarding.
- feature-home: profile status and primary actions.
- feature-capture: sheet manifest, drawing canvas, validation, progress.
- feature-font-studio: generation status, preview, font settings.
- feature-keyboard: IME service, key layout, input actions, keyboard settings.
- feature-export: composer, renderer, PNG output and Sharesheet.
- core-data: Room entities/DAO, DataStore, file repository, migrations.
- core-font: glyph normalization, outline conversion, font compiler interface, validation.
- core-design: shared components, spacing, colors, typography, icon style.

Font engine boundary

Define a stable interface such as generateFont(profileId, settings) returning a candidate asset plus validation results. The implementation must run off the main thread, report progress, support cancellation, and never replace the active font before validation. Keep the font engine independently testable with fixed glyph fixtures.

Performance targets

- Drawing input should feel immediate; do not perform font compilation inside touch callbacks.
- Database and file writes should run off the main thread where appropriate.
- Font generation should show real progress stages rather than a fake percentage.
- Long previews and exports must have memory limits and handle cancellation.
- The app should launch into a usable state without waiting for expensive font regeneration.

18. Testing strategy

Unit tests

- Glyph normalization preserves relative shape and baseline.
- Empty and invalid glyphs are handled correctly.

- Unicode mapping covers every required code point.
- Font candidate validation detects missing tables and missing required glyphs.
- File promotion is atomic and retains last valid font after failure.
- Capture progress is derived correctly from saved glyphs.
- Settings survive process restart.

Integration tests

- Capture a sample alphabet, generate TTF, load it in preview, and render a sample sentence.
- Kill the app during capture and generation; verify recovery.
- Enable and select the IME through Android settings and test typing in app-owned fields.
- Test IME in multiple third-party apps and common EditText types.
- Verify password fields do not enable learning or content logging.
- Export PNG and share it through the Android Sharesheet.
- Test light/dark modes, keyboard resize, rotation, insets, and narrow screen layouts.

Manual release checks

- Fresh install and first-run onboarding.
- Upgrade install without data loss.
- Device reboot and force-stop recovery.
- No internet connection during all core flows.
- Low-storage handling.
- TalkBack and large text pass.
- No visual clipping or off-screen controls.

19. Definition of done

A feature is complete only when its real behavior, persistence, error handling, and UI are implemented and tested. A screen with static mock data is not done. A button with no action is not done. A font generation animation without a valid TTF is not done.

- All visible controls have meaningful actions or are clearly disabled with an explanation.
- No mock profile, fake capture progress, or fabricated font status is used in production.
- All user-created data persists across app restarts.
- Font generation output is validated before activation.
- Keyboard is a real Android IME and passes the editing test matrix.
- The app is usable offline and requests only necessary permissions.
- PNG output is accurate, unclipped, and shareable.
- All major screens have empty, loading, success, and failure states.
- Build and lint checks pass; no critical crash remains in the release flow.

20. Phased delivery plan

Phase	Deliverable	Exit criterion
0 — Feasibility spike	Tiny glyph capture + vector outline + real TTF + Android preview	A generated font renders a test sentence on Android.
1 — App foundation	Native app, navigation, theme, splash, onboarding, local persistence	Restart-safe profile and capture flow.
2 — Full capture	All required English glyph sheets, validation, resume, replacement	Complete capture manifest can be saved and resumed.
3 — Font Studio	Generation, validation, preview, settings, versioned font assets	Valid font can be regenerated without losing previous asset.

4 — Complete keyboard	IME layouts, editor actions, settings integration, privacy-safe behavior	Keyboard works in test fields and common apps.
5 — Handwriting Studio	Text composer, paper/ink controls, PNG rendering and sharing	Shareable PNG renders correctly offline.
6 — Hardening	Accessibility, error handling, performance, release testing, legal notices	Acceptance checklist passes on target devices.

Do not begin Phase 4 before Phase 0 validates the font pipeline. UI work can proceed in parallel after the initial proof, but avoid polishing a false success path.

21. Risks and mitigations

Risk	Mitigation
On-device font compilation is more complex than expected	Validate a minimal pipeline first; keep compiler behind an interface and choose an Android-compatible approach based on a working prototype.
Single glyph samples look repetitive	Set expectations; add alternate glyph capture and contextual substitutions in a later phase.
Handwriting looks too thin or too heavy	Use outline generation and preview-based stroke presets; do not merely scale a bitmap.
Apps do not show the font	Explain Unicode behavior early and make PNG export a first-class companion feature.
Keyboard trust concerns	No network requirement, no text logging, transparent disclosure, and password-field safeguards.
Data loss during generation	Temporary candidate files, validation, atomic promotion, and last-known-good asset.
AI coding tools create fake functionality	Use phase-specific prompts, acceptance tests, inspect generated native files, and require real device validation.

22. Suggested build prompt for an AI coding tool

Use this as the project master instruction. Then implement one phase at a time; do not ask the coding tool to generate everything in one pass.

```
Build InkKey, a complete native Android app in Kotlin with Jetpack Compose. It is a zero-budget, offline-
first personalized handwriting keyboard. The user captures English letters and punctuation using guided
one-glyph-at-a-time drawing cells; the app stores vector strokes locally, generates and validates a real
TTF font using a proven Android-compatible font pipeline, previews the font, and provides a complete
Android InputMethodService QWERTY keyboard. Include a handwriting composer that renders text using the
generated font to a shareable PNG.

Non-negotiable: no paid API, mandatory account, cloud backend, fake font generation, mock persistence, fake
keyboard activation, or UI-only buttons. All user data must persist. Use Room for metadata, DataStore for
preferences, app-private files for glyphs and fonts, versioned font assets, atomic promotion, and
preservation of the last valid font. Use InputConnection for keyboard editing and respect EditorInfo/IME
actions. Do not log or transmit typed text; add password-field safeguards. The keyboard sends Unicode text
and must not claim that other apps will render the custom font. The visual handwriting export is the
reliable way to share custom-font output.

Design: premium, clean, light-by-default, carefully aligned, consistent spacing, accessible touch targets,
polished empty/loading/error/success states, responsive layouts, and no clipped controls. Implement splash
on every cold launch, onboarding, Home, guided Capture Studio, Font Studio, keyboard enable/select/test
flow, Handwriting Studio, Settings, privacy/help, and local data deletion. Externalize UI strings for
future localization. Use free tooling only.

Start with Phase 0 feasibility: capture a tiny set of glyphs, compile a valid TTF, and render it on
```

```
Android. Do not proceed to full UI polish until the font pipeline is proven. Report exact files changed,
real functionality implemented, tests run, known limitations, and remaining work. Never mark a feature
complete unless it works on device.
```

Appendix A — Supported character manifest for v1

Lowercase: abcdefghijklmnopqrstuvwxyz

Uppercase: ABCDEFGHIJKLMNOPQRSTUVWXYZ

Digits: 0123456789

Punctuation: . , ? ! : ; ' " - ( )

Optional symbols: & / @ + = %

Unsupported glyphs must use a documented fallback or show a missing-glyph indicator in the font preview/export. Do not map unrelated characters to a visually misleading glyph.

Appendix B — Product language

Use	Avoid
“Create your handwriting font”	“AI clones your handwriting perfectly”
“Type normally with InkKey”	“Every app will show your handwriting”
“Share as a handwritten image”	“Send custom-font text that everyone sees identically”
“Your data stays on your device by default”	Absolute privacy claims that have not been verified against the shipped build